

Introduction

If you've spent any time building workflows in n8n, you've probably reached for the Code node the moment a built-in node couldn't quite do what you needed. It's the escape hatch — full JavaScript, full control. Which is exactly why it's also where the most confusing, hardest-to-notice bugs live.

Here's the scenario: fifty rows flow into a Code node. The workflow runs. Green checkmark, no errors, no warnings. And only one item comes out the other end. Forty-nine rows are just gone — not filtered, not errored, just silently absent from the output.

This isn't really an n8n problem. It's a mental-model problem, and it's one that trips up a lot of people coming from n8n's older Function node, or from any framework where "per-item" execution is automatic. Once you understand what the Code node actually hands you and what it expects back, this entire category of bug disappears — and you'll also start recognizing the pattern in other batch-processing systems, because "operate on a collection, not a single record" is a fundamental idea in software engineering, not an n8n quirk.

Problem Overview

In n8n, data moves between nodes as a list of items. A "Set All" or "Split In Batches" node, a database query, an API call returning multiple rows — all of these produce an array of items, not a single object.

The Code node, when set to run once for all items (the default and most common mode), receives that entire array in one execution. Your job inside the node is to read all of them, do something with each one, and hand back a properly shaped array containing all of them.

The bug happens when people assume the runtime will loop for them — calling their code once per item automatically, the way the old Function node used to behave for certain configurations. It won't. If you write code that only reads the first item, that's the only item you'll process. If you write code that returns raw values instead of properly wrapped items, n8n won't recognize your return value as data at all.

Example

Say you have fifty contact records flowing into a Code node, and you want to trim whitespace off each contact's `name` field.

Wrong approach:

```
const name = $json.name.trim();  
return [{ name }];
```

Output: one item. The other forty-nine names never got touched, and nothing in the UI tells you that.

Correct approach:

```
const items = $input.all();  
return items.map(item => ({  
  json: { name: item.json.name.trim() }  
}));
```

Output: fifty items, each with a trimmed name, each properly wrapped for the next node to consume.

The difference between these two snippets is exactly two things: where you read the data from, and how you shape what you return. Everything else about the code is the same.

Intuition

Before jumping into code, it helps to build the right picture in your head of what's actually happening.

Picture `$input` as a mail tray dropped on your desk — fifty unopened envelopes, all at once. `$json`, by contrast, is like someone handing you just the top envelope off that tray and calling it "the mail." It's not wrong, exactly — it is a piece of the mail — but it's not the whole delivery.

`$json` exists because early on, when you're prototyping a Code node, n8n's test panel feeds you a single pinned example item. Reading `$json.name` at that point looks completely correct, because there genuinely is only one item in front of you during development. The problem is invisible until real, multi-row data hits the node in production — at which point `$json` still quietly resolves to item zero, and every other row is left untouched.

An experienced engineer, seeing "I need to process a batch," reaches instinctively for three moves: get the full collection, iterate over it, and produce a new collection in the shape the next stage expects. That's the entire intuition. The Code node isn't special here — it's just JavaScript array processing wearing an n8n costume.

Brute Force Solution

There isn't really a "brute force vs. optimal" split here in the algorithmic sense — this isn't a performance problem, it's a correctness problem. But it's worth walking through the naive, incomplete first attempt, because it's exactly the trap most people fall into.

Idea: Treat the Code node like a single-record transform. Read `$json`, transform it, return it.

```
const name = $json.name.trim();
return [{ json: { name } }];
```

Why it seems fine: It runs. It doesn't error. During testing with one pinned item, it produces the exact expected output.

Why it fails: It only ever touches the first item in the batch. Every other row is silently discarded — not skipped with a warning, not logged, just absent from the node's output.

- **Advantage:** Simple, reads like ordinary single-object code.
- **Disadvantage:** Loses every item after the first. Fails silently in production with no error signal.
- **Time complexity:** $O(1)$ — which is precisely the bug, since it should be $O(n)$.
- **Space complexity:** $O(1)$.

Optimal Solution

The fix has exactly two parts, and both matter — doing only one of them still breaks your workflow.

Step 1 — Read the full batch. Use `$input.all()` instead of `$json`. This returns an array of every item flowing into the node, each shaped like `{ json: {...}, binary: {...} }`.

Step 2 — Return every item, properly wrapped. Whatever transform you're doing, map over the array and return a new array. Each entry in that array must be an object with a `json` key holding your actual data — not the raw data itself.

Step	What it does	Common mistake
Read	<code>\$input.all()</code> gets every item	Using <code>\$json</code> , which is just item 0
Transform	<code>.map()</code> over the array	Using <code>.forEach()</code> and forgetting to return
Return	Wrap each result as <code>{ json: {...} }</code>	Returning raw objects with no <code>json</code> key

Think of it as a pipeline: **unwrap the tray** → **do work on each envelope** → **reseal each envelope before sending it downstream**. Skip the resealing step and the next node either throws an error or misreads your data as garbage.

Python Code

The video is JavaScript-specific (n8n's Code node), but the identical pattern — read the full collection, transform each element, return a collection with correct structure — is worth internalizing in Python, since it's the same shape you'll hit in any batch-processing or ETL context.

```
def process_items(items: list[dict]) -> list[dict]:
    """
    Mirrors the n8n Code node pattern:
    read every item, transform each one, return every item.
    """
    return [
        {"json": {"name": item["json"]["name"].strip()}}
        for item in items
    ]

if __name__ == "__main__":
    sample_items = [
        {"json": {"name": " Alice "}},
        {"json": {"name": "Bob "}},
        {"json": {"name": " Cara"}},
    ]

    result = process_items(sample_items)

    assert len(result) == len(sample_items), "Item count changed – a row was dropped"
    assert all("json" in r for r in result), "Missing json wrapper on an item"
    assert [r["json"]["name"] for r in result] == ["Alice", "Bob", "Cara"]

    print("All items processed and wrapped correctly:", result)
```

Code Walkthrough

- `items: list[dict]` — stands in for what `$input.all()` returns: a list, one entry per incoming row.
- The list comprehension is the Python equivalent of `.map()` — it runs once per item and always produces a value for every input, so nothing gets skipped by accident the way a `for` loop with a missing `append` might.
- Each output entry is wrapped as `{"json": {...}}`, mirroring the exact shape n8n expects on the way out.
- The `__main__` block is a self-check: it asserts the item count didn't shrink and that every output item has the `json` wrapper — the two failure modes this whole article is about.

Dry Run

Input: three items — `" Alice "`, `"Bob "`, `" Cara"`.

1. List comprehension starts iterating.
2. Item 1: `" Alice ".strip()` → `"Alice"` → wrapped as `{"json": {"name": "Alice"}}`.
3. Item 2: `"Bob ".strip()` → `"Bob"` → wrapped similarly.
4. Item 3: `" Cara".strip()` → `"Cara"` → wrapped similarly.
5. Final list has three wrapped items — count matches input, nothing lost.
6. Assertions pass: length matches, every item has a `json` key, values are correctly trimmed.

Complexity Analysis

- **Time complexity:** $O(n)$, where n is the number of items. Each item is visited exactly once — correct, because you fundamentally cannot transform n items in less than n operations.
- **Space complexity:** $O(n)$, since the output is a new list the same size as the input. This is expected and correct — you're not being wasteful, you're required to produce one output record per input record.

The original buggy version was $O(1)$ in both time and space — which felt efficient, but was actually just incomplete work disguised as efficiency.

Alternative Solutions

If your Code node is only filtering rows or renaming a field, skip the Code node entirely and use n8n's built-in **Filter** node or **Set** node instead. Both operate correctly across the full item array by default — you avoid this entire class of bug because there's no manual loop to get wrong. Reach for the Code node specifically when you need real branching logic, external library calls, or transforms that the built-in nodes can't express.

Edge Cases

- **Empty input:** `$input.all()` returns `[]`. Your `.map()` correctly returns `[]` too — no crash, no phantom item.
- **Single item:** Works identically to multi-item; there's no special-casing needed, which is a sign the pattern is correct.

- **Missing field on one item:** `item.json.name` throws if `name` is `undefined` on any row — add a guard `( item.json.name ?? "" )` if your data isn't guaranteed clean.
- **Very large batches:** Still $O(n)$ — fine for typical workflow volumes, but if you're processing tens of thousands of items per execution, consider n8n's Split In Batches node to chunk the work.
- **Binary data attached to items:** `$input.all()` items can carry a `binary` key alongside `json` — if you only return `json`, you'll drop any attached files.

Common Mistakes

1. Reading `$json` inside a "run once for all items" Code node, silently limiting yourself to item zero.
2. Returning raw objects `( { name } )` instead of wrapped items `( { json: { name } } )`.
3. Using `.forEach()` for the transform and forgetting that it doesn't return anything, leaving you with `undefined` as your return value.
4. Trusting the test/pinned-data panel as proof the node works — it only ever shows you one item, so the bug is invisible until real data hits it.
5. Mutating `item.json` directly and returning `item` — this can work, but it obscures which fields changed and makes debugging harder than returning a clean new object.

Interview Questions

- "Why would a data transformation step silently drop records instead of throwing an error?"
- "Walk through the difference between `.map()`, `.forEach()`, and `.filter()` and when each is appropriate."
- "How would you defensively code a batch transform so a malformed record doesn't crash the whole batch?"
- "What's the tradeoff between processing items one-at-a-time versus operating on the full collection at once?"

Similar Problems

- **LeetCode 1. Two Sum** — same underlying skill: iterate a full collection rather than assuming a single value, since checking only the first element misses the answer.
- **LeetCode 118. Pascal's Triangle** — building a full result array row by row mirrors returning a fully-populated output collection instead of a partial one.

- **LeetCode 283. Move Zeroes** — another case where it's tempting to short-circuit after touching one element, when the correct solution requires visiting every element in the array.

Key Takeaways

The Code node isn't broken — it's doing exactly what you told it to do. `$json` giving you one item and `map()` needing a `json` wrapper are both documented behaviors; the bug is entirely in the mental model, not the tool. Two rules fix it permanently: read the full batch with `$input.all()`, and return every item wrapped as `{ json: {...} }`. And if you're not doing real logic, skip the Code node altogether — the Filter and Set nodes get you the same result without this failure mode.

Watch the Video

For the full walkthrough with a live before-and-after run showing forty-nine items vanishing and then all fifty coming through correctly, watch the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series that takes tools developers use every day — n8n, SQL, Python, system design, interview prep — and pulls them apart until the confusing parts actually make sense. No fluff, no assumed expertise, just the specific mental model that fixes the specific bug.

Call To Action

If this saved you from a silent data-loss bug, like the video on YouTube and subscribe for the rest of the series. For deeper written breakdowns like this one, subscribe to the Substack. Drop a comment with what you're stuck on in n8n right now — these lessons come directly from what people ask about.

Quick Review

n8n Code node: reading input — why not \$json?

`$json` only sees item 0. Use `$input.all()` to get every incoming item as an array.

Time complexity of processing \$input.all() items?

$O(n)$ — one pass over n incoming items to build the output array.

Space complexity of the working pattern?

$O(n)$ — new output array holds one entry per input item, plus originals in memory.

Common bug: Code node returns fewer items than expected?

Forgot the `{json:...}` wrapper shape per item — n8n needs an array of `{json:{...}}` objects, not raw values.

\$json vs \$input.all() — when to use each?

`$json`: single-item context (Item Lists mode). `$input.all()`: 'Run Once for All Items' mode needing every row.

Interview follow-up: why loop with map instead of forEach here?

`map` returns a new array matching required output shape; `forEach` returns undefined, breaking the node's return contract.

n8n Lesson 22: Code node: working with \$input and multiple items (reading all incoming items, returning a new items array correctly) — free Python cheat sheet